CHAPITRE 3

EXPLORER LES RÉSEAUX DE NEURONES AVEC TENSORFLOW PLAYGROUND

3. 1 Introduction

Comprendre le fonctionnement d'un réseau de neurones peut parfois sembler abstrait. Il est souvent difficile de visualiser comment les différents paramètres influencent l'apprentissage.

TensorFlow Playground est un outil interactif en ligne qui permet d'observer en temps réel l'effet de chaque paramètre sur le réseau de neurones. Il constitue un excellent moyen pour se familiariser avec le concept de réseau, sans avoir à coder.

Dans ce module, nous allons explorer certains paramètres essentiels et observer leur impact sur l'apprentissage et la capacité de généralisation du réseau. Les principaux paramètres que vous pouvez modifier sont :

Couches cachées : Ajouter ou retirer des couches pour voir comment cela affecte la profondeur du réseau.

Neurones par couche : Changer le nombre de neurones dans chaque couche pour observer la variation de la complexité.

Taux d'apprentissage (*learning rate*) : Ajuster la vitesse à laquelle le réseau apprend.

Nombre de cycles d'apprentissage (*epochs*) : Définir combien de fois le réseau parcourt l'ensemble des données.

Fonctions d'activation : Expérimenter avec différentes fonctions (sigmoid, relu, tanh) pour voir leur influence sur la sortie.

Techniques de régularisation (L1, L2) : Ajouter des contraintes pour éviter le surapprentissage.

Taille des lots (*batch size*) : Modifier le nombre d'exemples traités simultanément pour chaque mise à jour des poids.

Bruit dans les données : Ajouter du bruit pour tester la robustesse et la généralisation du réseau.

Objectif : Comprendre de manière concrète comment ces différents paramètres influencent l'apprentissage et la capacité du réseau à généraliser à de nouvelles données.
En manipulant ces paramètres dans TensorFlow Playground, vous pourrez visualiser immédiatement l'effet de chaque choix et développer une intuition solide sur le comportement des réseaux de neurones.

3. 2 Présentation de TensorFlow Playground

TensorFlow Playground est un outil web interactif développé par Daniel Smilkov et Shan Carter. Il permet de visualiser en temps réel l'effet des hyperparamètres sur un réseau de neurones, de manière graphique et intuitive, sans avoir besoin d'écrire de code complexe.

Voici quelques informations clés sur cet outil :

Type d'outil : application web interactive accessible à l'adresse https://playground.tensorflow.org.

Technologies utilisées : JavaScript et D3.js pour la visualisation graphique.

Limites : ne supporte que les réseaux séquentiels simples.

Avantages :

Permet d'expérimenter facilement avec différentes architectures de réseaux.

Visualise immédiatement l'impact des modifications sur la fonction de perte (*loss*) et les décisions du réseau.

Aide à développer une intuition sur le comportement des réseaux de neurones.

3. 2. 1 Interface et principaux contrôles

3. 2. 1. 1 Bandeau supérieur

Le bandeau supérieur de TensorFlow Playground permet de contrôler l'exécution du réseau et de modifier ses principaux paramètres. Les fonctionnalités principales sont :

Reset / Run / Pause / Step : réinitialiser le réseau, démarrer ou arrêter l'entraînement, ou exécuter une étape (*epoch*) à la fois.

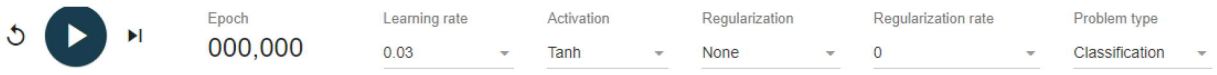
Epochs : nombre de cycles d'apprentissage effectués.

Learning Rate : taux d'apprentissage, c'est-à-dire le pas utilisé par l'algorithme de descente de gradient.

Activation : choisir la fonction d'activation des neurones (sigmoid, relu, tanh, etc.).

Regularization : appliquer une régularisation L1 ou L2 pour réduire le risque de surapprentissage.

Problem type : sélectionner le type de problème à résoudre : classification binaire, régression ou classification multi-classe.



figures 3.1: Bandeau supérieur de contrôle

3. 2. 1. 2 Section des entrées du réseau

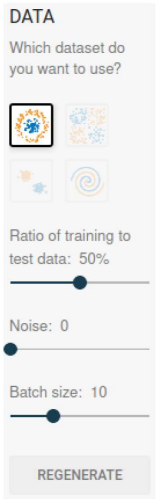
Cette section permet de configurer les données utilisées pour l'apprentissage et la validation. Les paramètres principaux sont :

Choix de la base de données : quatre modèles prédéfinis sont disponibles.

Ratio train/test : proportion des données utilisées pour l'entraînement et la validation.

Ajout de bruit (*noise*) : introduire des variations aléatoires dans les données pour tester la robustesse du réseau.

Taille des lots (*batch size*) : nombre d'exemples traités simultanément à chaque mise à jour des poids.



figures 3.2: Section d'entrée et sélection des features

3. 2. 1. 3 Architecture graphique du réseau

Dans TensorFlow Playground, le réseau de neurones est représenté de manière visuelle pour mieux comprendre son fonctionnement. Les éléments principaux sont :

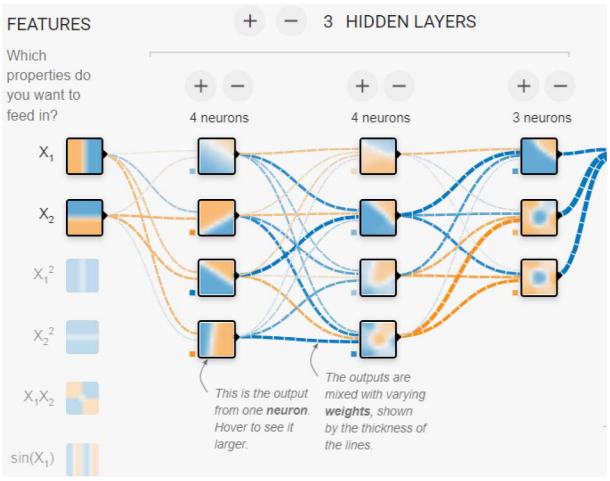
Couches : représentées verticalement, on peut ajouter ou retirer jusqu'à 8 couches cachées.

Neurones : le nombre de neurones par couche peut être modifié pour observer l'impact sur la complexité du réseau.

Poids : visualisés par l'épaisseur et la couleur des traits reliant les neurones.

Biais : indiqués par un petit carré noir à gauche de chaque neurone.

Sorties des neurones : représentées par des carrés colorés qui indiquent la valeur du neurone pour chaque point du domaine.



figures 3.3: Visualisation graphique des neurones et des poids

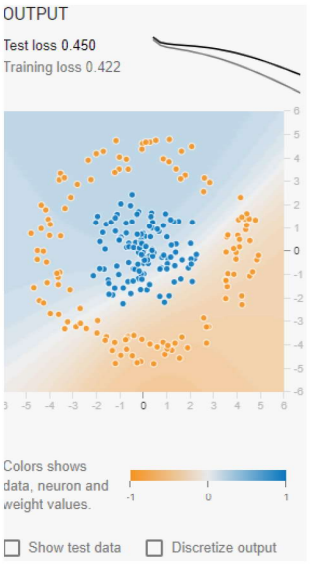
3. 2. 1. 4 Sortie et suivi de l'apprentissage

Cette section permet de suivre l'évolution de l'apprentissage et la qualité des prédictions du réseau :

Courbes de perte : *training loss* (gris) et *test loss* (noir) pour suivre l'erreur d'apprentissage.

Carte de sortie : superposée aux données d'entraînement pour visualiser la décision du réseau.

Discrétisation de la sortie : option *discretize output* pour mieux observer les frontières de décision.



figures 3.4: Sortie et perte du réseau

3. 3 Conseils pratiques pour expérimenter

Pour tirer le meilleur parti de TensorFlow Playground et comprendre les réseaux de neurones, suivez ces recommandations :

1. Commencez avec 0 couche cachée (perceptron simple) pour observer les limites d'un modèle linéaire.
2. Ajoutez progressivement des neurones et des couches pour explorer les séparations non linéaires.
3. Changez la fonction d'activation (*sigmoid*, *tanh*, *ReLU*) et observez leur influence sur l'apprentissage.
4. Testez différents taux d'apprentissage pour voir l'effet sur la vitesse et la stabilité de convergence.
5. Expérimentez avec la régularisation (L1/L2) et le *dropout* pour comprendre leur impact sur la généralisation.
6. Ajoutez du bruit aux données pour tester la robustesse du réseau.
7. Essayez différentes bases de données (spirale, cercles, XOR) pour observer comment l'architecture du réseau doit s'adapter.

Astuce pédagogique : Passez la souris sur les poids et les neurones pour voir leurs valeurs exactes et comprendre comment les décisions se forment pendant l'apprentissage.

3. 4 Conclusion

TensorFlow Playground est un outil interactif idéal pour :

Visualiser le processus d'apprentissage d'un réseau de neurones.

Comprendre l'impact des hyperparamètres sur les performances.

Expérimenter facilement avant de coder un réseau complet.

Il rend l'apprentissage du Deep Learning beaucoup plus concret et intuitif pour les débutants.

CHAPITRE 4

MES PREMIERS PAS EN DEEP LEARNING AVEC KERAS

4. 1 Keras: La librairie de deep learning python

4. 1. 1 Introduction

Keras est une API de réseau de neurones écrite en Python, peut être utilisé sur l' un des trois librairies disponibles: Tensorflow, Théano et CNTK. Par défaut, Keras utilise la librairie TensorFlow en tant que librairie de manipulation de tenseurs. Les instructions pour configurer une autre librairie se trouvent dans https://keras.io/backend/. C'est une abstraction de haut niveau de ces cadres d'apprentissage en profondeur et rend donc l'expérimentation plus rapide et plus facile. Keras est modulaire, ce qui signifie que la mise en œuvre est transparente, les développeurs pouvant rapidement étendre des modèles en ajoutant des modules.
TensorFlow est une bibliothèque de logiciels open-source pour l'apprentissage automatique. Cela fonctionne efficacement avec le calcul impliquant des tableaux; c'est donc un excellent choix pour le modèle que nous allons construire dans ce tutoriel. De plus, TensorFlow permet l'exécution de code sur un processeur CPU ou un processeur graphique GPU, ce qui est une fonctionnalité utile, en particulier lorsque vous travaillez avec un jeu de données volumineux.

4. 1. 2 Caractéristiques de Keras

1. **Convivialité**. Keras est une API développée pour les humains, pas pour les machines. Elle met l'expérience utilisateur au centre de tout. Keras suit les meilleures pratiques afin de réduire la charge cognitive : elle propose une API simple et cohérente, minimisant le nombre d'actions requises pour les cas les plus communs, et les messages d'erreur sont suffisamment explicites pour permettre une résolution simple des problèmes.

2. **Modularité**. Un modèle est considéré comme une séquence ou un graph de modules autonomes, complètement configurables et qui peuvent être reliés les uns aux autres avec le minimum de restrictions possible. En particulier, les couches de neurones, les fonctions de coût, les optimiseurs, fonctions d'activation, mécanismes de régularisation, sont tous des modules indépendants que l'on peut combiner pour créer de nouveaux modèles.

3. **Extensibilité**. Il est facile d'ajouter de nouveaux modules, en tant que classes ou fonctions, et les modules existants fournissent de nombreux exemples. La simplicité de création de nouveaux modules permet une expression totale, ce qui fait de Keras un outil parfaitement adapté à la recherche avancée.

4. **Fonctionnement en Python**. Pas de fichiers séparés de configuration des modèles en format déclaratif. Tout est déclaré dans le code, ce qui permet un débogage facile et facilite l'extensibilité.

4. 2 Installation

Keras utilise les dépendances suivantes:

numpy, scipy

Pyyaml

HDF5 et h5py (facultatif, requis si vous utilisez des fonctions de sauvegarde / chargement de modèle)

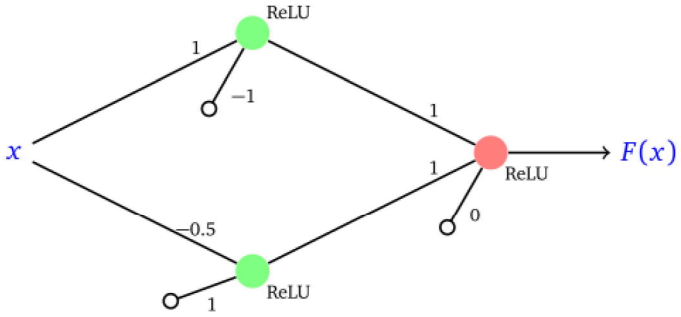
NVIDIA cuDNN (recommandé si vous souhaitez utiliser Keras sur un GPU). Les instructions pour installer cuDNN se trouvent dans https://docs.nvidia.com/deeplearning/sdk/cudnn-install/

scikit-image (facultatif, requis si vous utilisez les fonctions intégrées de keras pour le prétraitement et l'augmentation des données d'image).

Je ne vais pas détailler comment installer python, tensorflow et keras. Si vous n'avez pas déjà python 3.7 installé sur votre machine, je vous conseille d'utiliser Anaconda (https://www.anaconda.com/products/individual) pour créer un environnement python 3.7 et ensuite suivre les étapes décrites dans le page https://www.tensorflow.org/install/ pour installer tensorflow sur votre machine et finalement un *pip install keras* pour installer également la bibliothèque keras(https://keras.io/).

4. 3 Comment construire un RNA déjà paramétré?

Le module Python tensorflow est très puissant pour l'apprentissage automatique. Le module keras a été élaboré pour pouvoir utiliser tensorflow plus simplement. Dans cette partie nous utilisons la partie facile : ***comment construire un réseau de neurones déjà paramétré?***
Le module keras permet de définir facilement des réseaux de neurones en les décrivant couche par couche. Pour l'instant nous définissons les poids à la main, en attendant de voir plus tard comment les calculer à la machine. Pour commencer nous allons créer le réseau de neurones correspondant à la figure suivante :



4. 3. 1 Module keras de tensorflow

En plus d'importer le module numpy (abrégé par np), il faut importer le sous-module keras du module tensorflow et quelques outils spécifiques :

```
from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense,Activation
```

4. 3. 2 Architecture du réseau

On va définir l'architecture d'un réseau très simple(voir la figure), en le décrivant couche par couche.

```
 modele = Sequential()
# Couches de neurones
modele.add(Dense(2, input_dim=1, activation= 'relu' ))
modele.add(Dense(1, activation= 'relu' ))
```

Explications :

La classe Sequential est une structure, initialement vide, qui permet de définir un empilement de couches de neurones.

Notre réseau s'appelle modele, il est du type Sequential, c'est-à-dire qu'il va être décrit par une suite de couches les unes à la suite des autres.

Chaque couche est ajoutée par une commande : modele.add(Dense(nb_neurones, activation=ma_fonction))

Une couche de type Dense signifie que chaque neurone de la nouvelle couche est connecté à toutes les sorties des neurones de la couche précédente.

Pour chaque couche, il faut préciser le nombre de neurones qu'elle contient. S'il y a n neurones alors la couche renvoie n valeurs en sortie. On rappelle qu'un neurone renvoie la même valeur de sortie vers tous les neurones de la couche suivante.

Pour la première couche, il faut préciser le nombre de valeurs en entrée (par l'option *input_dim =...)*. Ici on a une entrée d'une seule variable.

Pour les autres couches, le nombre d'entrées est égal au nombre de sorties de la couche précédente. Il n'est donc pas nécessaire de le préciser.

Pour chaque couche, il faut également préciser une fonction d'activation (c'est la même pour tous les neurones de cette couche).

Notre exemple ne possède qu'une entrée et comme il n'y a qu'un seul neurone sur la dernière couche alors il n'y a qu'une seule valeur en sortie.

Une fois mis en place, on obtient des informations sur le modèle par la commande :

```
print(modele.summary())
```

qui fournit un résumé chiffré du réseau avec le nombre de couches, le nombre de neurones par couche et le nombre total de poids (coefficients et biais) à calculer.
Vous pouvez alors voir la configuration de votre modèle par la commande :

```
print(modele.get_config())
```

4. 3. 3 Les poids

Lors de la définition d'un réseau et de la structure de ses couches, des poids aléatoires sont attribués à chaque neurone. La démarche habituelle est ensuite d'entraîner le réseau, automatiquement, afin qu'il trouve de **bons** poids. Pour l'instant, nous continuons de fixer les poids de chaque neurone à la main. La commande pour fixer les poids est:

```
set_weights()
```

Voici quelques précisions concernant de cette commande. Son utilisation n'est pas très aisée.

Les poids sont définis pour tous les éléments d'une couche, par une commande *set_weights(poids)*.

Les poids sont donnés sous la forme d'une liste : poids = [coeff,biais].

Les biais sont donnés sous la forme d'un vecteur de biais (un pour chaque neurone).

Les coefficients sont donnés sous la forme d'un tableau à deux dimensions. Il sont définis par entrée. Attention, la structure n'est pas naturelle (nous y reviendrons).

Voici les poids de la couche, numéro 0 et 1 :

```
# Couche 0
coeff = np.array([[1.,-0.5]])
biais = np.array([-1,1])
poids = [coeff,biais]
modele.layers[0].set_weights(poids)
# Couche 1
coeff = np.array([[1.0],[1.0]])
biais = np.array([0])
poids = [coeff,biais]
modele.layers[1].set_weights(poids)
```

Pour vérifier que les poids d'une couche sont corrects, on utilise la commande *get_weights()*, par exemple pour la première couche :

```
modele.layers[0].get_weights()
```

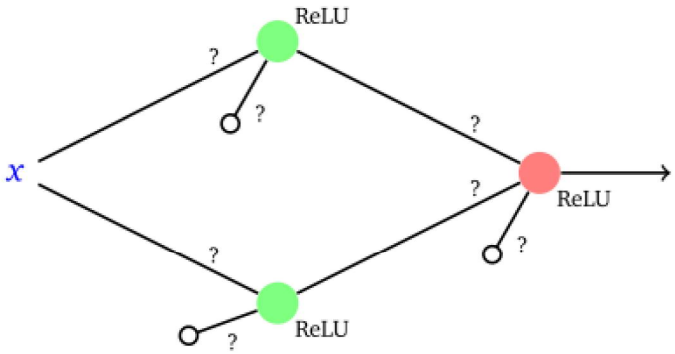
Cette instruction renvoie les poids sous la forme d'une liste [coefficients,biais] du type : [[[1. -0.5]], [-1. 1.]] Astuce! Cette commande est aussi très pratique avant même de fixer les poids, pour savoir quelle est la forme que doivent prendre les poids afin d'utiliser *set_weights()*.

4. 3. 4 Évaluation

Comment utiliser le réseau ? C'est très simple avec *predict()*. L'entrée correspond donc à un réel et la sortie également. Voici comment faire :

```
entree = np.array([[3.0]])
sortie = modele.predict(entree)
```

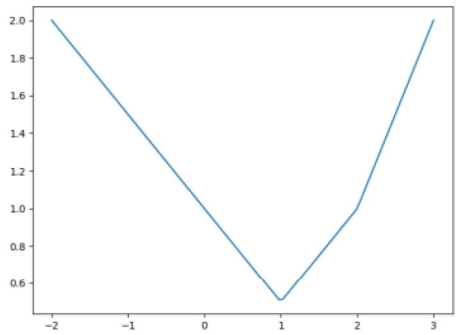
Ici sortie vaut [[2.0]] et donc F(3)=2. Ce que l'on peut vérifier à la main en calculant les sorties de chaque neurone.



4. 3. 5 Visualisation

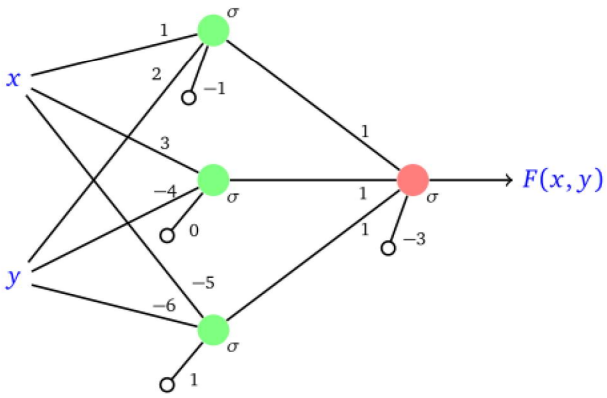
Afin de tracer le graphe de la fonction F : $\mathbb{R} \to \mathbb{R}$, on peut calculer d'autres valeurs :

```
import matplotlib.pyplot as plt
liste_x = np.linspace(-2, 3, num=100)
entree = np.array([[x] for x in liste_x])
sortie = modele.predict(entree)
liste_y = np.array([y[0] for y in sortie])
plt.plot(liste_x,liste_y)
plt.show()
```



4. 3. 6 Autre exemple

Créons le réseau de neurones ci-dessous :



Couches

```
# Architecture du réseau
modele = Sequential()
# Couches de neurones
modele.add(Dense(3, input_dim=2, activation= 'sigmoid' ))
modele.add(Dense(1, activation= 'sigmoid' ))
```

La première couche possède 3 neurones, chacun ayant deux entrées. La seconde couche n'a qu'un seul neurone (qui a donc automatiquement 3 entrées). La fonction d'activation est partout la fonction σ.

Poids

```
# Couche 0
coeff = np.array([[1.0,3.0,-5.0],[2.0,-4.0,-6.0]])
biais = np.array([-1.0,0.0,1.0])
poids = [coeff,biais]
modele.layers[0].set_weights(poids)
# Couche 1
coeff = np.array([[1.0],[1.0],[1.0]])
biais = np.array([-3.0])
poids = [coeff,biais]
modele.layers[1].set_weights(poids)
```

Remarquez que les poids ne sont pas définis neurone par neurone, mais par entrée : d'abord les poids de la première entrée pour chaque neurone, puis les poids de la seconde entrée pour chaque neurone, etc.

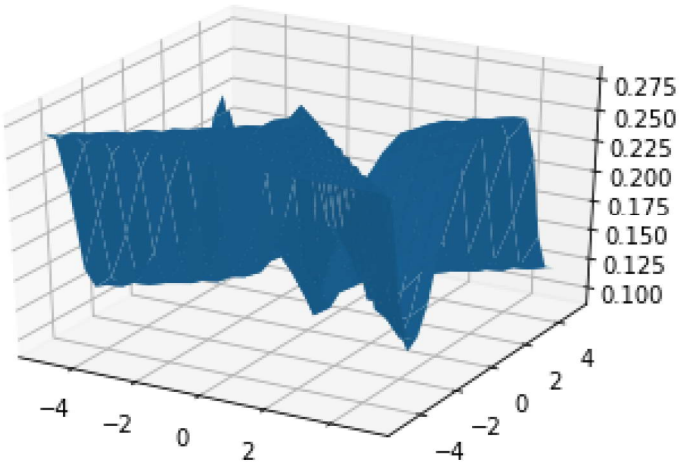
Évaluation

```
entree = np.array([[7,-5]])
sortie = modele.predict(entree)
```

Cette fois l'entrée est du type (x, y) et la sortie correspond à un seul réel F (x, y). Ici $F(7,-5) \simeq 0.123$.
Voici comment tracer le graphe de F : $\mathbb{R}^2 \to \mathbb{R}$. C'est un peu trop technique, ce n'est pas la peine d'en comprendre les détails.

Graphique

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
VX = np.linspace(-5, 5, 20)
VY = np.linspace(-5, 5, 20)
X,Y = np.meshgrid(VX, VY)
entree = np.c_[X.ravel(), Y.ravel()]
sortie = modele.predict(entree)
Z = sortie.reshape(X.shape)
fig = plt.figure()
ax = plt.axes(projection= '3d ' )
ax.plot_surface(X, Y, Z)
plt.show()
```


0.275
0.250
0.225
0.200
0.175
0.150
0.125
0.100
−4
−2
0
2
−4
−2
0
2
4